CCE 2 Unit 4 Report

1. Project Title: Develop an order ID management system using an AVL tree to display each node's balance factor and verify whether the tree satisfies AVL properties

2. Problem Statement:

The Order ID Management System is designed to manage order IDs efficiently using an AVL Tree data structure in C++. In order management applications, storing, searching, and displaying order IDs efficiently is an important task.

Traditional data structures, such as arrays and ordinary Binary Search Trees (BSTs), may have limitations when handling frequently changing data. In an ordinary BST, inserting elements in an unfavorable order can create an unbalanced tree, increasing the time required for searching and insertion.

To overcome this problem, this project uses an AVL Tree, which is a self-balancing Binary Search Tree. Each node stores a unique order ID, its height, and pointers to its left and right child nodes.

The system allows users to insert order IDs, search for a particular order, display all order IDs along with their balance factors, and verify whether the tree satisfies AVL properties.

The main purpose of this project is to demonstrate the practical application of AVL Trees and understand how self-balancing data structures improve the efficiency of order management.

3. **Objectives:**

1. To Develop an Order ID Management System

The main objective is to develop a simple and efficient Order ID Management System using the C++ programming language.

2. To Implement an AVL Tree

The project aims to understand and implement an AVL Tree for storing and managing order IDs efficiently.

3. To Insert Order IDs

The system allows users to insert unique order IDs into the AVL Tree. The tree automatically performs rotations whenever necessary to maintain balance.

4. To Search for Order IDs

The system enables users to search for a specific order ID and determine whether it exists in the tree.

5. To Display Balance Factors

The system displays each node's order ID, height, and balance factor to help users understand the structure and balance of the AVL Tree.

4. **Theory:**

The Order ID Management System is an application developed using the AVL Tree data structure in C++. It is used to store, search, and display order IDs while maintaining the balance of the tree.

An AVL Tree is a self-balancing Binary Search Tree (BST). Each node contains an order ID, its height, and pointers to its left and right child nodes.

In an AVL Tree, the balance factor of a node is calculated by subtracting the height of its right subtree from the height of its left subtree.

Balance Factor = Height of Left Subtree - Height of Right Subtree

The balance factor of every node in a valid AVL Tree must be -1, 0, or +1.

When a new order ID is inserted, the system updates the heights and balance factors of the affected nodes. If a node becomes unbalanced, the system performs suitable rotations, such as Left Rotation, Right Rotation, Left-Right Rotation, or Right-Left Rotation.

The system also provides a verification operation that checks the balance factor of every node to determine whether the tree satisfies AVL properties.

Thus, this project demonstrates the practical application of AVL Trees, rotations, searching, insertion, and balance-factor verification in C++.

5.Algorithm:

Step 1: Start the program.

Step 2: Initialize an empty AVL Tree.

Step 3: Display the menu with the following operations:

- Insert Order ID
- Display Orders and Balance Factors
- Verify AVL Properties
- Search Order ID
- Count Total Orders
- Exit

Step 4: Accept the user's choice.

Step 5: If the user selects Insert Order ID, accept an order ID and insert it into the AVL Tree.

Step 6: Update the heights and balance factors of the affected nodes.

Step 7: If any node becomes unbalanced, perform the appropriate AVL rotation.

Step 8: If the user selects Display Orders, traverse the tree and display each order ID, its height, and its balance factor.

Step 9: If the user selects Verify AVL Properties, check whether the balance factor of every node lies between -1 and +1.

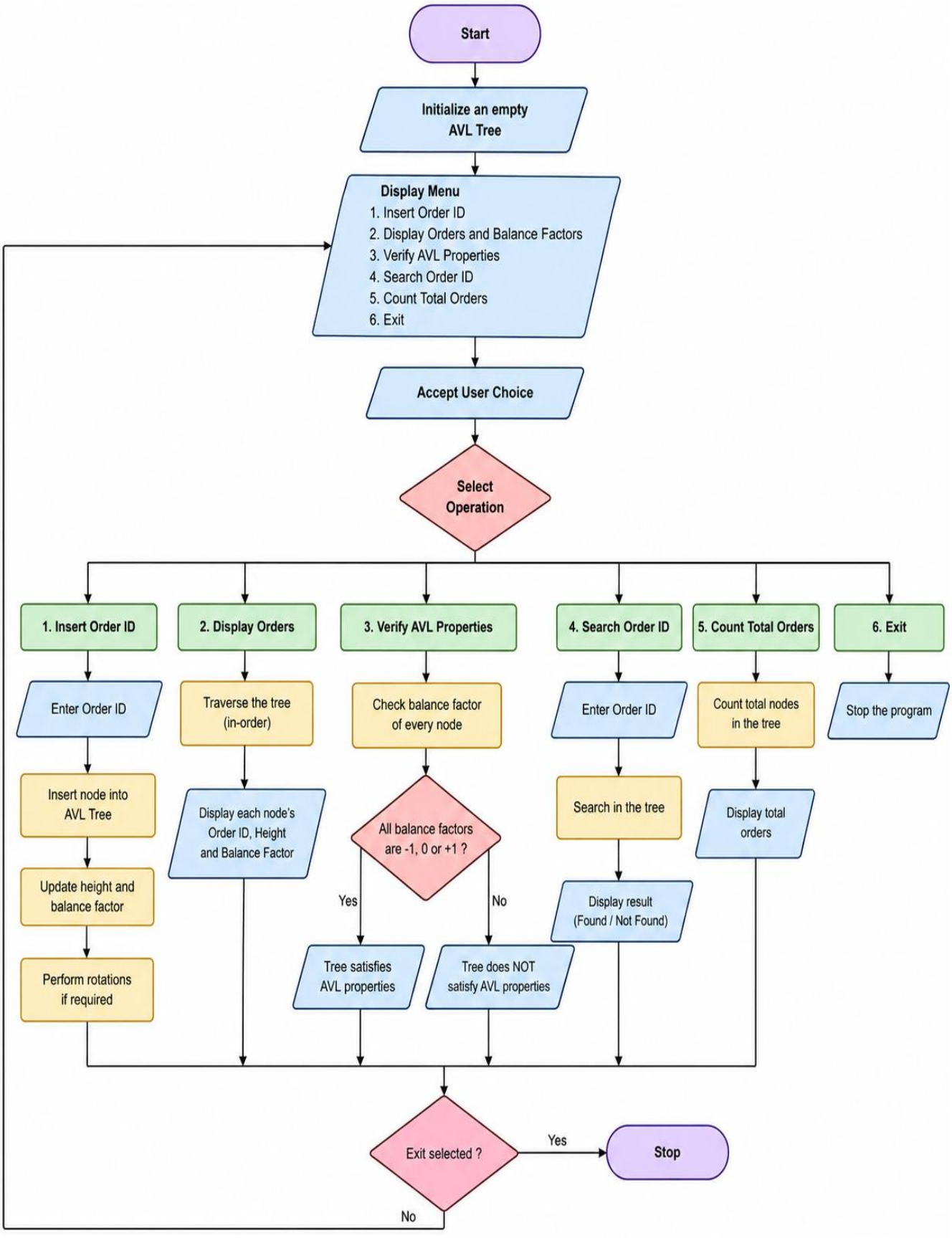
Step 10: If the user selects Search Order ID, search for the specified order ID and display whether it is found.

Step 11: If the user selects Count Total Orders, count and display the total number of nodes in the tree.

Step 12: Repeat Steps 3–11 until the user selects Exit.

Step 13: Stop the program.

6. Flowchart:


Start
Initialize an empty AVL Tree
Display Menu
1. Insert Order ID
2. Display Orders and Balance Factors
3. Verify AVL Properties
4. Search Order ID
5. Count Total Orders
6. Exit
Accept User Choice
Select Operation
1. Insert Order ID
2. Display Orders
3. Verify AVL Properties
4. Search Order ID
5. Count Total Orders
6. Exit
Enter Order ID
Insert node into AVL Tree
Update height and balance factor
Perform rotations if required
Traverse the tree (in-order)
Display each node's Order ID, Height and Balance Factor
Check balance factor of every node
All balance factors are -1, 0 or +1 ?
Yes
No
Tree satisfies AVL properties
Tree does NOT satisfy AVL properties
Enter Order ID
Search in the tree
Display result (Found / Not Found)
Count total nodes in the tree
Display total orders
Stop the program
Exit selected ?
Yes
Stop
No

7. Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Node {
    int orderID, height;
    Node *left, *right;

    Node(int id) {
        orderID = id;
        height = 1;
        left = right = nullptr;
    }
};

int height(Node* n) {
    return n ? n->height : 0;
}

int balanceFactor(Node* n) {
    return n ? height(n->left) - height(n->right) : 0;
}

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* t = x->right;

    x->right = y;
    y->left = t;

    y->height = 1 + max(height(y->left),
```

```
                    height(y->right));
    x->height = 1 + max(height(x->left),
                    height(x->right));

    return x;
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* t = y->left;

    y->left = x;
    x->right = t;

    x->height = 1 + max(height(x->left),
                    height(x->right));
    y->height = 1 + max(height(y->left),
                    height(y->right));

    return y;
}

Node* insert(Node* root, int id) {
    if (root == nullptr)
        return new Node(id);

    if (id < root->orderID)
        root->left = insert(root->left, id);
    else if (id > root->orderID)
        root->right = insert(root->right, id);
    else {
        cout << "Duplicate Order ID!\n";
        return root;
```

```
    }
    root->height = 1 + max(height(root->left),
                           height(root->right));

    int bf = balanceFactor(root);

    // LL Case
    if (bf > 1 && id < root->left->orderID)
        return rightRotate(root);

    // RR Case
    if (bf < -1 && id > root->right->orderID)
        return leftRotate(root);

    // LR Case
    if (bf > 1 && id > root->left->orderID) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    // RL Case
    if (bf < -1 && id < root->right->orderID) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}
void display(Node* root) {
    if (!root)
        return;

    display(root->left);
    cout << "Order ID: " << root->orderID
```

```
        << " | Height: " << root->height
        << " | Balance Factor: "
        << balanceFactor(root) << endl;
    display(root->right);
}
bool verifyAVL(Node* root) {
    if (!root)
        return true;
    int bf = balanceFactor(root);
    if (bf < -1 || bf > 1)
        return false;
    return verifyAVL(root->left) &&
           verifyAVL(root->right);
}
bool search(Node* root, int id) {
    if (!root)
        return false;
    if (root->orderID == id)
        return true;
    if (id < root->orderID)
        return search(root->left, id);
    return search(root->right, id);
}
void countOrders(Node* root, int &count) {
    if (!root)
        return;
    count++;
    countOrders(root->left, count);
    countOrders(root->right, count);
}
void showTotalOrders(Node* root) {
    int count = 0;
    countOrders(root, count);
```

```
    cout << "Total Orders: " << count << endl;
}

int main() {
    Node* root = nullptr;
    int choice, id;
    do {
        cout << "\n===== ORDER MANAGEMENT =====\n";
        cout << "1. Insert Order ID\n";
        cout << "2. Display Orders\n";
        cout << "3. Verify AVL Properties\n";
        cout << "4. Search Order ID\n";
        cout << "5. Count Total Orders\n";
        cout << "6. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter Order ID: ";
            cin >> id;
            root = insert(root, id);
            break;

        case 2:
            if (root == nullptr)
                cout << "No orders available!\n";
            else
                display(root);
            break;

        case 3:
            if (verifyAVL(root))
```

```
            cout << "Tree satisfies AVL properties.\n";
        else
            cout << "Tree violates AVL properties.\n";
        break;
    case 4:

        cout << "Enter Order ID to search: ";
        cin >> id;
        if (search(root, id))
            cout << "Order ID found!\n";
        else
            cout << "Order ID not found!\n";
        break;
    case 5:

        showTotalOrders(root);
        break;

    case 6:
        cout << "Exiting program...\n";
        break;

    default:
        cout << "Invalid choice!\n";
    }

  } while (choice != 6);

  return 0;
}
```

8. Difference Between Array and Linked List:

Sr. No.	Binary Search Tree(BST)	AVL Tree
1	An array stores elements in contiguous memory locations.	A linked list stores elements in nodes connected through pointers.
2	It may become unbalanced.	It may become unbalanced.
3	It does not automatically perform balancing rotations.	It uses rotations to maintain balance
4	Its balance factor is not restricted.	Every node must have a balance factor of -1, 0, or +1.
5	Searching may take O(n) time in the worst case.	Searching takes O(log n) time.
6	Insertion may take O(n) time in the worst case.	Insertion takes O(log n) time.
7	It is simpler to implement.	It requires height and balance-factor management.

9. Result:

The Order ID Management System using an AVL Tree was developed using C++. The system provides a menu-driven interface that allows users to perform different order management operations.

The system successfully performs the following operations:

- **Insert Order ID:** New order IDs can be inserted into the AVL Tree.
- **Search Order ID:** Users can search for a specific order ID.
- **Display Orders:** All order IDs are displayed in sorted order along with their heights and balance factors.
- **Verify AVL Properties:** The system checks whether the tree satisfies the AVL balance condition.
- **Automatic Balancing:** Rotations are performed whenever required to maintain the AVL Tree structure.
- **Count Total Orders:** The system displays the total number of order IDs stored in the tree.

The project provides practical knowledge of AVL Trees, insertion, searching, rotations, height calculation, and balance-factor verification in C++.

10. Conclusion

The Order ID Management System using an AVL Tree is a useful project developed in C++ to manage order IDs efficiently.

The AVL Tree maintains balance through rotations, allowing insertion and searching operations to be performed efficiently. The system also displays the balance factor of each node and verifies whether the tree satisfies AVL properties.

Through this project, we gain practical knowledge of self-balancing Binary Search Trees, tree traversal, rotations, height calculation, and balance-factor verification.

Overall, this project demonstrates the practical application of AVL Trees in order management and provides a foundation for developing more advanced order processing systems.